

IMPLEMENTASI ALGORITMA HIERARCHICAL CLUSTERING ANALYSIS DENGAN PEMROSESAN PARALEL UNTUK KLASTERISASI DATA PENONTON BIOSKOP

Ahmad Yadan Ulya Munaji ¹⁾, M. Ismail Cahyadi ²⁾ Yoga Adiguna ³⁾, ZAKY ⁴⁾.

Program Studi Teknik Informatika Universitas Mataram

Corresponding author e-mail: penulis@gmail.com (Times New Roman, 10 pt)

Abstrak

Perkembangan teknologi komputasi pada era digital mendorong kebutuhan akan pengolahan data dalam jumlah besar secara cepat dan efisien. Salah satu metode yang sering digunakan dalam proses pengelompokan data adalah Hierarchical Clustering Analysis (HCA). Metode ini mampu mengelompokkan data berdasarkan tingkat kemiripan antar objek tanpa perlu menentukan jumlah kluster di awal proses. Namun, penerapan HCA pada dataset berskala besar memiliki kelemahan berupa tingginya kompleksitas komputasi dan meningkatnya waktu eksekusi akibat proses perhitungan matriks jarak serta penggabungan kluster yang dilakukan secara berulang. Untuk mengatasi permasalahan tersebut, penelitian ini menerapkan pendekatan komputasi paralel menggunakan Message Passing Interface (MPI). Proses komputasi dibagi ke beberapa prosesor agar proses clustering dapat dijalankan secara simultan dan lebih efisien. Pengujian dilakukan dengan variasi jumlah proses mulai dari 4 hingga 22 proses serta variasi ukuran data yang terus meningkat. Hasil pengujian menunjukkan bahwa penggunaan pemrosesan paralel mampu meningkatkan efisiensi komputasi pada jumlah proses tertentu. Waktu eksekusi tercepat diperoleh pada penggunaan 6 proses dengan waktu sekitar 29 detik, sedangkan peningkatan jumlah proses secara berlebihan justru menyebabkan waktu eksekusi meningkat hingga mencapai sekitar 120 detik pada 22 proses. Selain itu, peningkatan ukuran data juga menyebabkan kenaikan waktu eksekusi secara signifikan. Hasil tersebut menunjukkan bahwa terdapat titik optimal pada penggunaan jumlah proses terhadap ukuran data tertentu agar performa sistem dapat bekerja secara maksimal. Dengan demikian, implementasi MPI pada algoritma HCA terbukti mampu meningkatkan efisiensi proses clustering, namun tetap dipengaruhi oleh keseimbangan antara jumlah data dan jumlah proses yang digunakan.

Kata kunci: Data Mining, Hierarchical Clustering Analysis (HCA), Komputasi Paralel, Message Passing Interface (MPI).

Abstract (Times New Roman10 pt)

The development of computing technology in the digital era has increased the need for fast and efficient processing of large-scale data. One method commonly used in data grouping processes is Hierarchical Clustering Analysis (HCA). This method is capable of grouping data based on the similarity level between objects without requiring the number of clusters to be determined at the beginning of the process. However, the implementation of HCA on large-scale datasets has drawbacks in the form of high computational complexity and increased execution time due to repeated distance matrix calculations and cluster merging processes. To overcome these problems, this study applies a parallel computing approach using the Message Passing Interface (MPI). The computational process is distributed across multiple processors so that the clustering process can be executed simultaneously and more efficiently. Testing was conducted using variations in the number of processes ranging from 4 to 22 processes and variations in increasing data sizes. The experimental results show that the use of parallel processing is able to improve computational efficiency at certain process configurations. The fastest execution time was obtained using 6 processes with an execution time of approximately 29 seconds, while excessive increases in the number of processes actually caused the execution time to rise to around 120 seconds when using 22 processes. In addition, increasing the amount of data also significantly increased execution time. These results indicate that there is an optimal point between the number of processes and the data size in order to achieve maximum system performance. Therefore, the implementation of MPI in the HCA algorithm has proven effective in improving clustering efficiency, although it is still influenced by the balance between data size and the number of processes used.

Keyword: Data Mining, Hierarchical Clustering Analysis (HCA), Parallel Computing, Message Passing Interface (MPI).

I. PENDAHULUAN

Industri perfilman dan bioskop mengalami perkembangan yang signifikan seiring dengan meningkatnya kebutuhan masyarakat terhadap hiburan. Dalam kondisi tersebut, pengelolaan data penonton menjadi aspek penting bagi pengelola bioskop untuk memahami perilaku, preferensi, serta pola kunjungan pelanggan. Informasi seperti usia, genre film yang diminati, frekuensi menonton, serta waktu kunjungan dapat dimanfaatkan sebagai dasar dalam penyusunan strategi bisnis, seperti penjadwalan film, promosi yang lebih tepat sasaran, hingga peningkatan kualitas layanan. Namun, tanpa pengolahan data yang sistematis, potensi informasi yang terkandung dalam data tersebut tidak dapat dimanfaatkan secara optimal dalam mendukung pengambilan keputusan.

Dalam praktiknya, pengelolaan data penonton menghadapi berbagai tantangan, terutama terkait dengan jumlah data yang terus meningkat dan keragaman karakteristik data. Pertumbuhan data yang pesat menyebabkan proses analisis menjadi kurang efisien apabila dilakukan dengan metode konvensional. Selain itu, belum adanya proses segmentasi yang terstruktur menyulitkan pengelola dalam mengidentifikasi kelompok penonton dengan karakteristik tertentu. Kondisi ini berdampak pada kurang optimalnya strategi pemasaran serta layanan yang diberikan, karena keputusan yang diambil belum sepenuhnya berbasis pada pola data yang akurat.

Salah satu pendekatan yang dapat digunakan untuk mengatasi permasalahan tersebut adalah teknik klasterisasi dalam data mining, khususnya menggunakan algoritma Hierarchical Clustering Analysis (HCA). Metode ini mampu mengelompokkan data berdasarkan tingkat kemiripan tanpa memerlukan penentuan jumlah klaster di awal [7], [8]. Melalui pendekatan ini, pola-pola tersembunyi dalam data penonton dapat diidentifikasi secara lebih sistematis, seperti segmentasi berdasarkan preferensi genre film atau intensitas kunjungan. Dengan demikian, HCA memberikan keunggulan dalam menghasilkan pemahaman yang lebih mendalam terhadap karakteristik penonton.

Meskipun demikian, penerapan HCA pada dataset berukuran besar memiliki keterbatasan, terutama dari sisi kompleksitas komputasi yang tinggi dan waktu proses yang

relatif lama [8]. Oleh karena itu, diperlukan pendekatan tambahan berupa pemrosesan paralel untuk meningkatkan efisiensi kinerja algoritma. Dengan memanfaatkan pemrosesan paralel, proses klasterisasi dapat dilakukan secara simultan pada beberapa unit komputasi, sehingga mampu mempercepat waktu eksekusi tanpa mengurangi akurasi hasil pengelompokan yang diperoleh [5], [6]. Kombinasi antara HCA dan pemrosesan paralel diharapkan dapat menghasilkan metode analisis yang lebih efektif dan efisien dalam mengolah data penonton bioskop.

Beberapa penelitian sebelumnya menunjukkan bahwa Hierarchical Clustering Analysis (HCA) dapat digunakan secara efektif untuk berbagai kebutuhan segmentasi data. Maori dan Mulyo [1] menerapkan Agglomerative Hierarchical Clustering untuk menganalisis tingkat kepuasan mahasiswa terhadap layanan akademik, sedangkan Rahmah et al. [3] menggunakan Hierarchical Clustering dalam pengelompokan provinsi berdasarkan kasus Demam Berdarah Dengue (DBD). Hasil penelitian tersebut menunjukkan bahwa HCA mampu menghasilkan pengelompokan yang representatif sesuai karakteristik data yang dianalisis. Namun, sebagian besar penelitian masih lebih berfokus pada kualitas hasil klasterisasi, sementara pembahasan mengenai efisiensi komputasi ketika jumlah data semakin besar masih relatif terbatas. Oleh karena itu, diperlukan pendekatan komputasi paralel untuk meningkatkan kinerja HCA pada proses pengolahan data berskala besar.

Berdasarkan latar belakang tersebut, penelitian ini bertujuan untuk mengimplementasikan algoritma Hierarchical Clustering Analysis yang dipadukan dengan pemrosesan paralel dalam proses klasterisasi data penonton bioskop. Penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan metode analisis data yang mampu menangani volume data besar secara optimal, serta menghasilkan informasi yang lebih akurat sebagai dasar dalam pengambilan keputusan strategis di industri perfilman.

II. TINJAUAN PUSTAKA

Clustering merupakan salah satu teknik data mining yang digunakan untuk mengelompokkan data berdasarkan tingkat kemiripan karakteristik

antar objek. Salah satu metode clustering yang banyak digunakan adalah Hierarchical Clustering Analysis (HCA), yaitu metode yang membentuk struktur hierarki dalam bentuk dendrogram sehingga hubungan antar objek dapat diamati secara bertingkat. Keunggulan utama HCA dibandingkan metode clustering lainnya adalah kemampuannya mengelompokkan data tanpa perlu menentukan jumlah kluster pada tahap awal proses. Penelitian yang dilakukan oleh Maori dan Mulyo menunjukkan bahwa Agglomerative Hierarchical Clustering mampu menghasilkan pengelompokan yang baik pada analisis tingkat kepuasan mahasiswa terhadap layanan akademik melalui pendekatan pengukuran jarak antar objek [1]. Pada penelitian lain, Maulana membandingkan beberapa metode linkage pada Agglomerative Hierarchical Clustering. Hasil penelitian menunjukkan bahwa Average Linkage menghasilkan struktur kluster yang relatif stabil sehingga sesuai digunakan pada data yang memiliki karakteristik beragam [2].

Penerapan Hierarchical Clustering juga telah digunakan pada berbagai kasus pengelompokan data di Indonesia, Rahmah et al. membandingkan metode Hierarchical Clustering dan K-Means dalam pengelompokan provinsi berdasarkan kasus Demam Berdarah Dengue (DBD) dan menunjukkan bahwa Hierarchical Clustering mampu memberikan hasil pengelompokan yang baik berdasarkan karakteristik wilayah yang diamati [3]. Hasil serupa juga ditunjukkan oleh Abdulpatah yang menerapkan Agglomerative Hierarchical Clustering untuk pengelompokan daerah penghasil padi di Indonesia dan memperoleh hasil kluster yang mampu menggambarkan karakteristik produksi antar wilayah secara lebih jelas [4]. Penelitian-penelitian tersebut menunjukkan bahwa HCA memiliki kemampuan yang baik dalam proses segmentasi data pada berbagai domain.

Meskipun demikian, salah satu kelemahan utama HCA adalah tingginya kompleksitas komputasi yang diperlukan selama proses pembentukan matriks jarak dan penggabungan kluster secara berulang. Kompleksitas tersebut menyebabkan waktu komputasi meningkat seiring bertambahnya ukuran data sehingga implementasi HCA pada dataset besar menjadi kurang efisien. Oleh karena itu, diperlukan pendekatan komputasi

paralel untuk mengurangi beban komputasi yang muncul selama proses clustering.

Salah satu teknologi yang banyak digunakan dalam komputasi paralel adalah Message Passing Interface (MPI). MPI merupakan standar komunikasi yang memungkinkan beberapa proses bekerja secara simultan dan saling bertukar data melalui mekanisme message passing. MPI menyediakan berbagai operasi komunikasi seperti Broadcast, Scatter, Gather, dan Reduce yang memungkinkan distribusi beban kerja secara efisien pada lingkungan komputasi paralel [5], [6]. Melalui pendekatan tersebut, proses perhitungan yang membutuhkan waktu tinggi dapat dibagi ke beberapa prosesor sehingga waktu eksekusi dapat dikurangi secara signifikan.

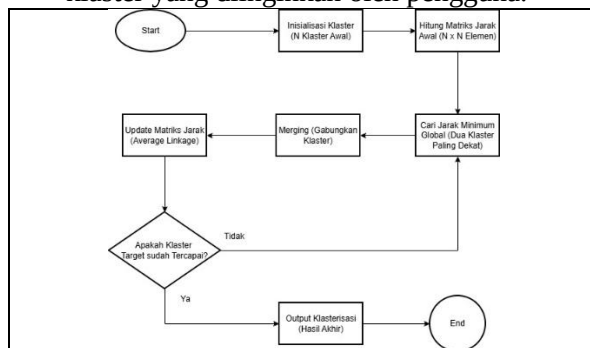
Sebagian besar penelitian sebelumnya lebih banyak membahas kualitas hasil pengelompokan yang dihasilkan oleh HCA, sementara itu pembahasan mengenai efisiensi komputasi khususnya ketika jumlah data semakin besar, masih belum banyak dilakukan, padahal kebutuhan komputasi menjadi salah satu tantangan utama dalam implementasi HCA pada dataset berukuran besar. Selain itu, implementasi Hierarchical Clustering Analysis berbasis MPI pada studi kasus segmentasi data penonton bioskop masih jarang ditemukan. Oleh karena itu, penelitian ini mengimplementasikan algoritma Hierarchical Clustering Analysis menggunakan MPI untuk mengevaluasi pengaruh jumlah proses terhadap efisiensi waktu komputasi pada proses klusterisasi data penonton bioskop.

III. METODOLOGI PENELITIAN

Algoritma *Hierarchical Clustering Analysis* (HCA) jenis *agglomerative* secara umum bekerja dengan prinsip *bottom-up*. Alur kerja konseptual dari algoritma ini dilakukan melalui tahapan-tahapan sebagai berikut:

- Inisialisasi Kluster: Setiap objek tunggal di dalam dataset mula-mula dianggap sebagai satu kluster mandiri yang terpisah dan unik. Jika terdapat N buah data, maka pada tahap awal akan terbentuk N buah kluster aktif.
- Perhitungan Matriks Jarak: Menghitung jarak kedekatan antar-seluruh pasangan objek data menggunakan metrik jarak tertentu (seperti *Euclidean distance*) untuk membentuk sebuah matriks jarak awal berukuran $N \times N$.

- Pencarian Jarak Minimum: Memindai seluruh isi matriks jarak untuk menemukan pasangan kluster aktif yang memiliki nilai jarak paling minimum atau memiliki tingkat kemiripan tertinggi.
- Penggabungan Kluster (*Merging*): Menggabungkan dua kluster terdekat yang telah ditemukan menjadi satu kelompok atau kluster baru. Proses ini mengakibatkan jumlah total kluster aktif di dalam sistem akan berkurang satu.
- Pembaruan Matriks Jarak: Menghitung kembali nilai jarak dari kluster baru yang terbentuk terhadap kluster-kluster lain yang masih aktif menggunakan metode tautan (*linkage*). Baris dan kolom milik kluster lama yang sudah bergabung akan dinonaktifkan.
- Kondisi Berhenti: Seluruh siklus pencarian jarak minimum, penggabungan, dan pembaruan matriks jarak di atas diulangi secara terus-menerus hingga semua objek data menyatu menjadi satu kluster besar tunggal, atau proses dihentikan saat jumlah kelompok telah mencapai target jumlah kluster yang diinginkan oleh pengguna.



Gambar 1. Flowchart Algoritma HCA Secara Umum

Penelitian ini termasuk dalam jenis eksperimental komputasi yang bertujuan mengimplementasikan algoritma *Hierarchical Clustering Analysis* (HCA) agglomerative dengan kriteria *Average Linkage* pada pemrosesan paralel menggunakan MPI (*Message Passing Interface*). Eksperimen dilakukan untuk mengukur kinerja algoritma dari sisi waktu eksekusi dan *speedup* pada berbagai ukuran data serta jumlah prosesor, serta mengevaluasi kualitas segmentasi penonton bioskop yang dihasilkan.

Data yang digunakan adalah data buatan (*synthetic data*) penonton bioskop yang

dibangkitkan secara acak. Setiap penonton direpresentasikan oleh 12 fitur numerik:

1. Usia : 15 – 70 tahun (bilangan bulat)
2. Pendapatan bulanan : Rp500.000 – Rp20.000.000
3. Frekuensi kunjungan : 1 – 10 kali per bulan

Data mentah tersebut dihasilkan dengan *random seed* yang sama untuk setiap ukuran dataset agar perbandingan antar skenario berjalan adil. Menggunakan data tersebut, variasi yang dijalankan akan berdasarkan jumlah data, jumlah prosesor, dan jumlah dimensi.

Algoritma HCA *agglomerative* bekerja secara *bottom-up*. Setiap objek (penonton) mula-mula dianggap sebagai satu kluster mandiri. Selanjutnya, dua kluster dengan jarak terkecil digabung secara iteratif hingga diperoleh sejumlah kluster yang diinginkan (dalam penelitian ini 5 kluster). Jarak antar kluster dihitung menggunakan kriteria *Average Linkage*, yang merata-ratakan jarak seluruh anggota dari kedua kluster, sehingga memberikan hasil yang seimbang dan tidak sensitif terhadap *outlier*. Jarak antar objek dihitung dengan rumus Euclidean distance yang diimplementasikan pada data terkategori. Secara matematis, jika ada dua objek penonton, misalnya objek x dan objek y , yang masing-masing memiliki sejumlah fitur numerik atau data dari kategorisasi, maka rumus Jarak Euclidean didefinisikan sebagai berikut:

$$d(x, y)^n = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

Di mana:

- $d(x, y)$ adalah nilai jarak geometris (kedekatan) antara objek penonton x dan objek penonton y .
- d merupakan jumlah dimensi atau fitur data yang digunakan dalam perhitungan (dalam penelitian ini bervariasi dari 3 hingga 12 dimensi).
- x_i adalah nilai numerik dari objek penonton x pada dimensi/fitur ke i .
- y adalah nilai numerik dari objek penonton y pada dimensi/fitur ke i .

Sebelum rumus ini diterapkan, seluruh fitur data mentah penonton (usia, pendapatan bulanan, dan frekuensi kunjungan) terlebih dahulu melalui tahap kategorisasi dan normalisasi lokal di

setiap rank prosesor. Proses ini krusial agar perbedaan skala satuan (misalnya nilai pendapatan yang mencapai jutaan rupiah dibandingkan dengan usia yang hanya puluhan tahun) tidak mendominasi hasil perhitungan jarak, sehingga setiap fitur memiliki bobot yang adil dalam menentukan kemiripan antar-penonton.

Setelah jarak antar-objek ditentukan, algoritma memperbarui jarak antar-klaster yang terbentuk. Penghitungan jarak antar-klaster dalam penelitian ini dilakukan dengan kriteria *Average Linkage* yang memiliki tahapan sebagai berikut:

- Konsep Dasar: Metode ini menentukan jarak antara dua klaster berdasarkan nilai rata-rata dari seluruh kombinasi pasangan objek yang ada di dalam masing-masing klaster. Pendekatan ini dipilih karena menghasilkan struktur pengelompokan yang lebih stabil dan tidak sensitif terhadap pencilan (*outlier*).
- Formula Matematis: Misalkan terdapat klaster A dan klaster B yang telah bergabung menjadi satu klaster baru bernama klaster AB. Untuk menghitung jarak antara klaster baru AB dengan suatu klaster lain (C) yang berada di luar kelompok tersebut, digunakan rumus:

$$d(AB, C) = \frac{|A| d(A, C) + |B| d(B, C)}{|A| + |B|}$$

Keterangan :

- $d(AB, C)$ adalah nilai jarak baru antara klaster gabungan AB dengan klaster C.
- $d(A, C)$ adalah jarak antara klaster A dengan klaster C sebelum terjadi penggabungan.
- $d(B, C)$ adalah jarak antara klaster B dengan klaster C sebelum terjadi penggabungan.
- $|A|$ merupakan kardinalitas atau jumlah total anggota objek yang berada di dalam klaster A.
- $|B|$ merupakan kardinalitas atau jumlah total anggota objek yang berada di dalam klaster B.

Paralelisasi dilakukan pada dua bagian utama yang paling memakan waktu:

1. Pembangunan matriks jarak awal – menggunakan distribusi blok segitiga atas.

2. Pembaruan matriks jarak pada setiap iterasi penggabungan – menggunakan pembagian rentang indeks klaster.

Komunikasi antar proses diimplementasikan melalui fungsi-fungsi MPI standar (Scatter, Gather, Broadcast, Reduce). Dengan cara ini, beban komputasi dibagi secara merata ke sejumlah prosesor yang bekerja serentak. Terdapat 3 jenis data waktu yang dicari, perbandingan variasi jumlah data dari 200 hingga 2000 data terhadap 10 jumlah prosesor, perbandingan variasi jumlah prosesor dari 4,6,8, ... hingga 22 terhadap 1000 jumlah data, perbandingan variasi jumlah dimensi dari 3 hingga 12 terhadap 1000 dan 10 jumlah data dan node. Berikut skenario pengujiannya:

1. Inisialisasi dan Persiapan Data
 - a. Semua Rank: Inisialisasi MPI, mengetahui rank dan size.
 - b. Rank 0: Membangkitkan data mentah sebanyak N penonton (sesuai skenario: 200, 400, hingga 2000) dengan *random seed* tetap. Data mentah tersebut terdiri dari tiga kolom (tergantung berapa dimensi yang ingin digunakan, sebagai contoh 3 dimensi): usia, pendapatan bulanan, dan frekuensi kunjungan. Data mentah dibagi secara merata ke seluruh rank menggunakan Scatter.
 - c. Setiap Rank (termasuk Rank 0): Menerima subset data mentah dan melakukan normalisasi lokal (kategorisasi) dengan aturan:
 - i. Usia $\rightarrow$ 3 kategori: Remaja (<25), Dewasa (25-44), Paruh baya/Lansia ($\geq$ 45).
 - ii. Pendapatan $\rightarrow$ 2 kategori: Rendah (<Rp5.000.000), tinggi ($\geq$ Rp5.000.000).
 - iii. Frekuensi $\rightarrow$ 2 kategori: Jarang (<4 kali/bulan), sering ($\geq$ 4 kali/bulan).
 - d. Setiap Rank: Mengirim hasil kategorisasi lokal ke Rank 0 menggunakan Gather.
 - e. Rank 0: Menggabungkan semua hasil menjadi dataset lengkap terkategori, lalu menyiarkan dataset tersebut ke seluruh rank melalui broadcast.
2. Membuat Matriks Jarak Awal dengan Grid-Based Distribution.

- a. Rank 0: Menentukan ukuran grid G berdasarkan akar kuadrat dari dua kali jumlah proses, lalu membagi matriks jarak segitiga atas menjadi blok-blok kecil sejumlah $*G(G+1)/2*$.
 - b. Setiap Rank: Diberi sejumlah blok untuk dihitung. Di dalam setiap blok, dihitung jarak Euclidean antar pasangan penonton (berdasarkan data terkategori) dan dicatat pasangan dengan jarak minimum lokal.
 - c. Setiap Rank: Mengirimkan seluruh entri jarak dan minimum lokal ke Rank 0 menggunakan Gather dan Reduce.
 - d. Rank 0: Menyusun matriks jarak lengkap $N \times N$ dari semua entri yang diterima, serta menentukan jarak minimum global dari kumpulan minimum lokal.
 - e. Rank 0: Menyiarkan matriks jarak lengkap dan informasi minimum global ke semua rank melalui Broadcast.
3. Inisialisasi Kluster Aktif
 - a. Rank 0: Menginisialisasi setiap penonton sebagai satu kluster mandiri (total N kluster) dengan ukuran = 1. Membentuk daftar kluster aktif dan menyimpan keanggotaannya.
 - b. Rank 0: Menyiarkan daftar kluster aktif ke semua proses agar setiap rank memiliki informasi yang konsisten.
 4. Iterasi Penggabungan Kluster dengan Average Linkage
 - a. Semua Rank: Memulai loop hingga jumlah kluster mencapai target (5 kluster). Iterasi pertama menggunakan pasangan minimum global yang telah diperoleh dari langkah 2.
 - b. Rank 0: Pada setiap iterasi, menampilkan pasangan kluster yang akan digabung, menyimpan riwayat penggabungan, serta memperbarui ukuran dan anggota kluster.
 - c. Rank 0: Menonaktifkan kluster yang telah bergabung, lalu menyiarkan perubahan status (active, cluster_size, num_clusters) ke semua rank.
 5. Update Matriks Jarak dan Pencarian Jarak Minimum Paralel
 - a. Setiap Rank: Setelah penggabungan, setiap rank menghitung ulang jarak dari kluster baru (hasil gabungan) ke seluruh kluster aktif lainnya yang menjadi jatahnya. Perhitungan menggunakan rumus Average Linkage:

$$d(AB, C) = \frac{|A| \cdot d(A, C) + |B| \cdot d(B, C)}{|A| + |B|}$$
 - b. dengan $|A|$ dan $|B|$ adalah ukuran kluster sebelum digabung.
 - d. Setiap Rank: Mengirimkan hasil perhitungan jarak baru ke Rank 0 menggunakan Gather.
 - e. Rank 0: Memperbarui matriks jarak dengan nilai-nilai baru, mengisi baris/kolom kluster yang sudah tidak aktif dengan ∞ , lalu menyiarkan matriks terbaru ke semua rank.
 - f. Setiap Rank: Secara paralel mencari pasangan kluster dengan jarak minimum pada subset baris matriks yang menjadi tanggung jawabnya.
 - g. Setiap Rank: Mengirimkan minimum lokal (jarak dan indeks pasangan) ke Rank 0 menggunakan Gather.
 - h. Rank 0: Menentukan minimum global dari semua minimum lokal, kemudian menyiarkan pasangan tersebut untuk digunakan pada iterasi berikutnya.
 6. Penghapusan Kluster dan Pengecekan Kondisi Berhenti
 - a. Setiap Rank: Setelah menerima status baru, kluster yang telah digabung (nonaktif) diabaikan pada iterasi selanjutnya.
 - b. Setiap Rank: Memeriksa apakah num_clusters sudah mencapai target. Jika belum, iterasi dilanjutkan menggunakan pasangan minimum global yang baru dihitung.
 7. Visualisasi dan Analisis Hasil Akhir
 - a. Rank 0: Setelah iterasi selesai (tersisa 5 kluster), menampilkan matriks jarak final dalam bentuk heatmap.
 - b. Rank 0: Menghitung profil setiap kluster berdasarkan rata-rata data mentah asli (usia, pendapatan, frekuensi) untuk menginterpretasikan karakteristik segmen penonton, misalnya:
 - i. Kluster A: usia muda, pendapatan rendah, frekuensi

tinggi →
”Pelajar/Mahasiswa Aktif”

- ii. Kluster B: usia dewasa, pendapatan tinggi, frekuensi rendah → ”Professional Sibuk” dan seterusnya.
- c. Rank 0: Membuat visualisasi 3D scatter plot dengan sumbu usia, pendapatan, dan frekuensi, di mana warna titik menunjukkan keanggotaan kluster.
- d. Rank 0: Menampilkan rekomendasi strategi berbasis segmen.
- e. Rank 0: Mencatat dan menampilkan waktu eksekusi total untuk skenario yang dijalankan, sebagai bagian dari evaluasi kinerja (lihat penjelasan selanjutnya).

Untuk mengukur kinerja algoritma paralel, dirancang serangkaian skenario pengujian dengan tiga variabel yang bisa diatur:

1. Ukuran data (N): 200, 400, 600, 800, 1000, 1200, 1400, 1600, 1800, 2000 penonton.
2. Jumlah prosesor (p): 4, 6, 8, 10, 12, 14, 16, 18, 20, 22 proses.
3. Jumlah Dimensi (D): 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 Dimensi.

Setiap kombinasi (N, p, D) dijalankan dan dicatat waktu eksekusi total (dari selesai inisialisasi hingga clustering selesai). Perulangan dalam pengambilan data dilakukan sebanyak 3 kali lalu dicari hasil rata - rata-nya sebagai hasil yang sebenarnya dalam perulangan tersebut.

Metrik yang diukur:

1. Waktu Eksekusi (T): wall-clock time yang diukur dengan MPI.Wtime() pada Rank 0.
2. Speedup (S): dihitung dengan rumus
$$S = \frac{T_{serial}}{T_{parallel}(p)}$$
 Dimana T_{serial} adalah waktu pada 1 prosesor untuk N yang sama.
3. Efisiensi (E): $E = \frac{S}{p}$

Hasil pengukuran dianalisis untuk melihat bagaimana peningkatan jumlah prosesor memengaruhi kecepatan komputasi pada berbagai skala data. Data waktu dan *speedup* disajikan dalam bentuk tabel dan grafik garis untuk memperjelas tren.

IV. HASIL DAN PEMBAHASAN

A. Skenario Pengujian

Hasil pengujian kali ini kami menggunakan data sintetis untuk menjadi bahan dalam pembuatan tiga jenis grafik dalam tujuan menghitung pengaruhnya terhadap efisiensi waktu yang dihabiskan saat menjalankan HCA secara paralel menggunakan data yang disediakan. Ketiga grafik nantinya akan menampilkan data yang memiliki variasi data yang berbeda beda seperti jumlah data, jumlah node, dan jumlah dimensi. Jumlah data yang digunakan pada masing - masing grafik adalah sepuluh buah data yaitu data sejumlah 200 hingga 2000 (jarak setiap data adalah 200 data), sejumlah prosesor mulai dari 4 buah hingga 22 buah prosesor (jarak setiap data adalah dua prosesor), dan sejumlah dimensi mulai dari 3 dimensi hingga 12 dimensi.

Target kluster yang ditentukan adalah sebanyak lima kluster pada program dan nilai yang diambil untuk dimasukkan ke dalam tabel merupakan nilai hasil dari perhitungan rata - rata nilai yang dihasilkan dari 3 kali perulangan pada setiap program, fungsinya agar bisa mendapatkan nilai yang lebih stabil dibandingkan dengan hanya menjalankan kode sekali. Nilai yang didapatkan biasanya berbeda beda pada setiap perulangan, sehingga perlu diterapkan pengambilan nilai rata - rata untuk itu.

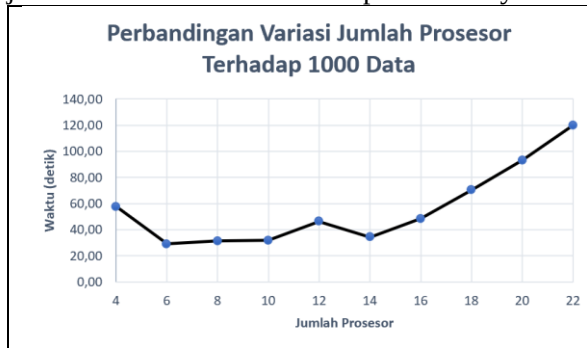
B. Hasil Pengujian



Gambar 2. Grafik Perbandingan Variasi Jumlah Data Terhadap 10 Prosesor

Berdasarkan **Gambar 2** di atas, terlihat tren yang terbentuk dari efisiensi waktu yang dihasilkan pada variasi jumlah data dari 200 hingga 2000 data semakin rendah efisiensi waktunya jika jumlah datanya semakin banyak. Artinya semakin banyak jumlah datanya maka efisiensi waktunya akan semakin berkurang, terlihat dari grafik yang semakin tinggi ketika jumlah data semakin banyak. Variasi data pada grafik tersebut menggunakan 10 buah prosesor

pada setiap jumlah datanya dengan tujuan agar perubahan tren ketika menggunakan variasi jumlah data bisa lebih terlihat perubahannya.



Gambar 3. Grafik Perbandingan Variasi Jumlah Prosesor Terhadap 1000 Data

Berdasarkan grafik hasil pengujian pada **Gambar 3**, terlihat adanya anomali performa di mana waktu eksekusi untuk beban kerja 1000 data justru melonjak hingga mencapai sekitar 120 detik ketika jumlah prosesor ditingkatkan hingga kapasitas maksimal 22 proses. Secara ilmiah, lambatnya waktu run-time pada jumlah prosesor yang banyak untuk ukuran data yang relatif kecil ini disebabkan oleh tingginya faktor *communication overhead* di dalam sistem paralel. Algoritma HCA berjalan secara interaktif dan sinkron di setiap iterasi penggabungan klaster, sehingga proses pembaruan matriks jarak global dan pencarian nilai minimum menuntut seluruh prosesor untuk saling bertukar data secara intensif menggunakan fungsi kolektif MPI seperti MPI_Scatter, MPI_Gather, dan MPI_Reduce. Ketika jumlah prosesor yang terlibat terlalu banyak sedangkan beban data lokal yang diproses sangat sedikit, waktu yang dihabiskan oleh infrastruktur sistem untuk melakukan jabat tangan (*handshaking*) dan mentransfer pesan lewat jaringan menjadi jauh lebih dominan daripada waktu komputasi riil untuk menghitung jarak objek itu sendiri.

Kondisi ini juga sangat berkaitan erat dengan masalah granularitas data yang terlalu halus (*fine-grained*) dan batasan efisiensi berdasarkan Hukum Amdahl. Sesuai dengan hukum tersebut, peningkatan kecepatan komputasi paralel selalu dibatasi oleh bagian sekuensial program yang tidak dapat dipecah ke prosesor lain, yang dalam implementasi ini berupa penentuan keputusan final kode klaster terdekat yang wajib diproses secara terpusat pada Rank 0. Akibatnya, alih-alih mempercepat pengerjaan, prosesor-prosesor yang banyak tersebut justru lebih sering berada dalam kondisi menganggur (*idle*) hanya untuk menunggu proses sinkronisasi data selesai dilakukan. Masalah ini kemudian diperparah oleh adanya ketidakseimbangan beban kerja (*load imbalance*) karena proses dekomposisi matriks jarak dalam penelitian ini hanya menghitung bagian segitiga atas matriks yang bersifat

simetris. Pada variasi prosesor yang besar seperti 22 proses, beberapa prosesor yang mendapatkan jatah blok grid kecil akan menyelesaikan komputasinya jauh lebih cepat dan terpaksa tertahan oleh pembatas komunikasi (*barrier*), yang pada akhirnya semakin memperburuk akumulasi waktu eksekusi total program secara keseluruhan.



Gambar 4. Grafik Perbandingan Variasi Jumlah Dimensi Terhadap 1000 Jumlah Data dan 10 Jumlah Node

Berdasarkan **Gambar 4** di atas, tren pada grafik tidak menunjukkan adanya efisiensi waktu yang benar – benar tinggi atau rendah. Data tren cenderung terlihat stabil tidak seperti tren pada dua grafik sebelumnya. Penambahan dimensi tidak terlalu mempengaruhi tren pada grafik seperti lonjakan nilai atau penurunan nilai efisiensi waktu. Meskipun terdapat kenaikan penurunan nilai pada efisiensi waktu setiap jumlah dimensi bertambah, namun penambahannya tidak terlalu signifikan, penambahannya cukup perlahan dibandingkan dengan yang lainnya, sehingga kesannya tidak terlalu mempengaruhi efisiensi waktu dalam menjalankan HCA secara paralel.

V. KESIMPULAN DAN SARAN

Berdasarkan hasil pengujian, dapat disimpulkan bahwa HCA memiliki titik optimalnya tersendiri untuk bisa memaksimalkan potensinya. Tidak seperti semakin banyak yang bekerja maka pengerjaan akan semakin cepat terselesaikan, namun bagaimana cara mengurangi resiko pekerja yang bisa menganggur dari pekerjaannya. Efektifitas waktu yang dihasilkan memiliki rentang nilai yang terkadang agak jauh sehingga perlu diberlakukan perhitungan rata – rata jika ingin mengambil data yang lebih baik dengan HCA. Semakin besar jumlah data yang diolah oleh HCA maka semakin lambat juga efektifitasnya. Jika data yang diberikan cukup besar, jumlah prosesor yang digunakan tidak semestinya mengikuti jumlah data yang semakin naik, karena semakin banyak prosesor semakin besar resiko pengangguran yang dilakukannya, sehingga tidak menuntut kemungkinan untuk

membuat efektifitas waktu yang dihasilkan semakin memburuk.

HCA secara keseluruhan cukup memberikan pengaruh yang signifikan terhadap penggunaan waktu dalam mengelola program. Dari data yang didapatkan, penghematan waktu bisa dihasilkan hingga lebih dari 50% dari yang aslinya. Hal itu akan sangat membantu dalam pengerjaan program, sehingga program bisa dijalankan dengan lebih cepat, tanpa perlu untuk menunggu terlalu lama.

VI. UCAPAN TERIMA KASIH (OPTIONAL)

Ditujukan untuk pihak-pihak yang mendukung proses penelitian (sumber dana, peran lainnya, hingga proses publikasi).

DAFTAR PUSTAKA

- [1] N. A. Maori and H. Mulyo, "Metode Single Linkage pada Agglomerative Hierarchical Clustering dalam Penentuan Tingkat Kepuasan Mahasiswa terhadap Layanan Akademik," *Simetris: Jurnal Teknik Mesin, Industri, Elektro dan Ilmu Komputer*, vol. 15, no. 2, 2024.
- [2] M. Y. Maulana, "Analysis of the Grouping of Provinces in Indonesia According to the Democracy Index With the Agglomerative Hierarchical Clustering Algorithm," *Jurnal Ilmu Komputer*, vol. 15, no. 2, pp. 27–35, 2022.
- [3] A. Rahmah, N. F. Khusna, S. A. Sanmas, S. Aulia, S. Amaria, and F. Fauzi, "Comparison Analysis of Hierarchical Clustering and K-Means Methods in Grouping Provinces in Indonesia Based on Dengue Hemorrhagic Fever Cases," *JUITA: Jurnal Informatika*, vol. 13, no. 2, 2025.
- [4] T. Abdulpatah, "Perbandingan Algoritma K-Means dan Agglomerative Hierarchical Clustering untuk Pengelompokan Daerah Penghasil Padi di Indonesia," *Jurnal Informatika dan Teknik Elektro Terapan*, vol. 13, no. 3, 2025.
- [5] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. Cambridge, MA: MIT Press, 1999.
- [6] P. S. Pacheco, *An Introduction to Parallel Programming*. San Francisco, CA: Morgan Kaufmann, 2011.
- [7] P.-N. Tan, M. Steinbach, and V. Kumar, *Introduction to Data Mining*, 2nd ed. Boston, MA: Pearson, 2019.
- [8] B. S. Everitt, S. Landau, M. Leese, and D. Stahl, *Cluster Analysis*, 5th ed. West Sussex, UK: Wiley, 2011.